

Business Rules — ParSU e-Feedback (BR-001–BR-100)

Provenance note: The original BR-001–BR-100 document was provided by the client during Phase 5 and is referenced throughout this codebase (model comments, service comments, integration test names, `docs/architecture.md`, `docs/schema-reconciliation.md`, `docs/testing/br-traceability.md`) but was never itself checked into this repository. The full text below is a **reconstruction**, compiled from every one of those in-repo references so the complete rule set exists in one place. Wording has been normalized into consistent rule statements; the *substance* of each rule is drawn directly from where it's implemented, tested, or discussed in this codebase. If you hold the original client document, treat that as authoritative and use this file to flag any discrepancy — don't assume this reconstruction is word-for-word identical to it.

Each rule below cites the primary file(s) that implement or test it, so any rule can be traced back to real code rather than taken on faith.

1. User Management

	Rule
BR-001	Each user account shall be assigned exactly one role: <code>student</code> , <code>office_staff</code> , <code>college_dean</code> , <code>qa_office</code> , or <code>administrator</code> .
BR-002	A single role may be assigned to multiple user accounts.
BR-003	Each user account shall have a unique email address.
BR-004	Passwords shall be stored as a hash only; the plaintext value is never persisted.
BR-005	A user account must be active (<code>isActive: true</code>) to authenticate.
BR-006	Each student account shall have a unique student number.
BR-007	Each staff/employee account shall have a unique employee ID.
BR-008	A student shall belong to exactly one college.
BR-009	Office staff and college deans shall belong to exactly one office (their service office or college).
BR-010	The administrator role has unrestricted access to all system functions — not "staff plus extra," a distinct permission set.
BR-011	Only administrators may create, update, deactivate, or delete staff/dean/QA/admin accounts. (Student accounts are the one exception — see the register-page scope decision below.)
BR-012	The system shall record a user's last successful login timestamp.
BR-013	An account shall be locked for a configurable duration after a configurable number of consecutive failed login attempts.

Sources: `src/models/User.ts`, `src/features/auth/services/auth.service.ts`, `src/features/admin/schemas/user.integration.test.ts`, `docs/schema-reconciliation.md` (register-page scope decision).

Reconciliation note (BR-011): read literally, BR-011 could be taken to forbid any self-service registration at all. Since the architecture blueprint already specified a public `/register` page and BR-008 only makes sense as something a student supplies about themselves at signup, this was reconciled as: BR-011 governs staff/dean/QA/admin accounts (provisioned by an admin from `/admin/users`); the public `/register` endpoint creates `role: "student"` accounts only and never accepts a role from the request body.

2–3. Colleges & Offices

	Rule
BR-014	A college is a distinct organizational unit that students and deans belong to.
BR-015	Each student belongs to exactly one college (same relationship as BR-008, stated from the college's side).
BR-016	Only administrators may create, update, or deactivate colleges.

	Rule
BR-017	An office may be associated with one or more complaint categories (via that category's default office / routing rules).
BR-018	An office may have one or more staff members assigned to it.
BR-019	Each office (including colleges, which share the same underlying collection) shall have a unique office code.
BR-020	Inactive offices shall not receive new complaint assignments.

Sources: `src/models/Office.ts`, `docs/schema-reconciliation.md`.

Design note: colleges are not a separate collection — they're `Office` documents with `type: "college"`. BR-014–016 don't require a distinct shape, and unifying the two avoids duplicating the same fields (name, code, active flag, hierarchy) twice.

4–6. Category / Routing / SLA

	Rule
BR-021	Every complaint belongs to exactly one category.
BR-022	Each category has exactly one fixed priority and one target office; both are mirrored onto the category record (<code>defaultPriority</code> , <code>defaultOfficeRef</code>) but are set from — and kept in sync with — that category's SLA rule and routing rule respectively, not edited on the category directly.
BR-023	Each category shall have at most one active routing rule at a time (enforced at the database level via a partial unique index).
BR-024	An inactive category cannot be selected when submitting a new complaint.
BR-025	A routing rule maps one category to one target office.
BR-026	The system shall automatically route a submitted complaint to the appropriate office based on its category (and optionally matching college/priority conditions) — no manual triage.
BR-027	Only administrators may create, update, or deactivate routing rules.
BR-028	A routing rule shall not reference an inactive office as its target.
BR-029	Each category shall have at most one active SLA configuration, fixed to that category's own priority (enforced via a partial unique index on <code>{ categoryRef }</code>); separately, at most one institution-wide default (<code>categoryRef: null</code>) may be active per priority level, used as a fallback for a category that has no SLA rule of its own.
BR-030	Each SLA configuration defines a first-response deadline.
BR-031	Each SLA configuration defines a resolution deadline (a separate clock from the response deadline).
BR-032	The system shall flag a complaint as overdue (<code>isOverdue</code>) once its resolution deadline has passed.
BR-033	A complaint approaching or breaching its SLA deadline shall be escalated to a designated office (<code>escalateToOfficeRef</code>), with a configurable warning threshold (default 80%) before the hard breach.
BR-034	Only administrators may create, update, or deactivate SLA configurations.

Sources: `src/models/Category.ts`, `src/models/RoutingRule.ts`, `src/models/SLARule.ts`, `src/features/routing-engine/services/routing.service.ts`, `src/features/sla/services/sla.service.ts`, `src/app/api/cron/sla-check/route.ts`.

Reconciliation note (BR-022/BR-029): the source material describes a category-level default office/priority and "one SLA configuration" per category. The admin UI collects a category's target office and priority at the routing-rule and SLA-rule steps respectively (not on the category form), and the server cascades those choices onto the category's `defaultOfficeRef` / `defaultPriority` fields — so a category's priority and target office each have exactly one place they're actually set, with the category record itself holding a read-only mirror. This is implemented as one active SLA rule per **category** (not per category + priority, since a category has only one priority), plus an optional institution-wide default per priority (`categoryRef: null`) for categories without a rule of their own.

7. Complaint Lifecycle

	Rule
BR-035	Only students may submit complaints.
BR-036	Each complaint shall be assigned a unique, human-readable, immutable ticket number (format <code>PREFIX-YYYY-NNNNNN</code> , e.g. <code>PARSU-2026-000001</code>).
BR-037	A complaint shall not be submitted without its required fields (category, title, description). Priority is not a submission field — it's inherited from the chosen category's fixed priority (BR-022), never chosen or edited by the student.
BR-038	Every complaint shall be linked to exactly one submitting student.
BR-039	A complaint shall be retrievable by its owning student.
BR-040	A complaint shall have exactly one current status at any time; a transition to the same status is rejected as a no-op.
BR-041	A complaint's status shall follow the defined lifecycle: <code>submitted</code> → <code>assigned</code> → <code>in_progress</code> → (⇌ <code>pending_information</code>) (⇌ <code>escalated</code>) → <code>resolved</code> → <code>closed</code> , with reopening permitted from <code>resolved</code> or <code>closed</code> back to <code>in_progress</code> .
BR-042	A closed complaint may not be edited by the student; its only valid transition is reopening.
BR-043	A resolved or closed complaint may be reopened by authorized personnel.
BR-044	The system shall track the number of times a complaint has been reopened (<code>reopenCount</code>), incrementing on every reopen cycle.
BR-045	Complaints shall never be deleted (no delete route exists); only administrators may archive a complaint (<code>isArchived</code>), which hides it from active views without removing the record.
BR-046	Each complaint shall record <code>submitted</code> , <code>resolved</code> , <code>closed</code> , <code>created</code> , and <code>updated</code> timestamps as five distinct fields.

Sources: `src/models/Complaint.ts`, `src/features/complaints/services/status-transitions.service.ts`,
`src/features/complaints/services/ticket-number.service.ts`, `src/features/complaints/complaints.integration.test.ts`.

8–10. Assignment / Timeline / Attachments

	Rule
BR-047	A complaint may have multiple assignment records over its lifetime, not just a single current assignee.
BR-048	Only authorized personnel (staff, dean, or admin) may assign or reassign a complaint.
BR-049	Each assignment record captures who assigned it, who it was assigned to, the source/destination office, and when.
BR-050	The initial assignment created by automatic routing is recorded as a system action (no human assigner — <code>assignedByRef: null</code>).
BR-051	Reassigning a complaint to a different office or staff member creates a new assignment record rather than modifying the previous one.
BR-052	Previous assignment records are never modified once created (append-only, enforced by only ever exposing a create operation).
BR-053	A timeline entry shall be created for every significant action on a complaint (submission, assignment, status change, note, attachment, escalation, resolution, reopening, closure, rating).
BR-054	Each timeline entry records the type of event that occurred.
BR-055	Each timeline entry records the actor responsible for it, or <code>null</code> for a system-generated event.
BR-056	Each timeline event's type is restricted to a defined, fixed set of event types.
BR-057	Timeline entries are immutable once created (append-only, same enforcement pattern as BR-052).
BR-058	A complaint may have multiple file attachments.
BR-059	Attachment records store only file metadata (URL, name, MIME type, size) — the schema has no field for binary content; actual files live in Cloudflare R2.
BR-060	Only files of a supported, configurable set of formats (MIME types) may be uploaded.
BR-061	Uploaded files shall not exceed a configurable maximum size.

	Rule
BR-062	Deleting an attachment does not delete its associated complaint.

Sources: `src/models/Assignment.ts`, `src/models/ComplaintTimeline.ts`, `src/models/Attachment.ts`, `src/features/attachments/attachments.integration.test.ts`, `src/app/api/uploads/presign/route.ts`, `docs/schema-reconciliation.md`.

Design note (BR-047): the original architecture blueprint kept only `assignedOfficeRef` / `assignedStaffRef` directly on `Complaint` — enough to know who has a ticket *now*, but not enough for "multiple assignment records throughout its lifecycle." A separate `assignments` collection was added in reconciliation; the fields on `Complaint` remain as cheap current-state pointers, with `assignments` as the immutable history behind them.

11–12. Notes / Ratings

	Rule
BR-063	Internal notes on a complaint are staff-facing only — never shown to the submitting student.
BR-064	A student-facing view of a complaint shall never expose internal notes or the flag that would reveal them.
BR-065	Each internal note belongs to exactly one complaint.
BR-066	Internal notes cannot be edited or deleted once created (no update route is ever wired up for this model).
BR-067	A student may submit exactly one satisfaction rating (1–5) per complaint.
BR-068	Only the submitting student may rate their own complaint.
BR-069	A rating may only be submitted once a complaint has reached <code>resolved</code> status.
BR-070	Both the rating value and an optional comment are recorded and persisted together.

Sources: `src/models/ComplaintNote.ts`, `src/models/Complaint.ts` (`studentRating` / `studentRatingComment`), `src/features/complaints/notes.integration.test.ts`, `src/features/complaints/rating.integration.test.ts`.

Design note (BR-067–070): ratings stay embedded on `Complaint` rather than becoming their own collection — bounded to exactly one per complaint, so embedding fits the same rationale used elsewhere in the schema.

13–14. Notifications / Audit Log

	Rule
BR-071	The system generates a notification record for defined trigger events (complaint submitted/assigned/status-updated/resolved, SLA warning, escalation, report generated).
BR-072	A single user may accumulate multiple notification records over time.
BR-073	A notification defaults to unread and can be marked read by its recipient.
BR-074	A notification's <code>type</code> is restricted to a defined, fixed set of event types.
BR-075	An audit log entry is written for administrative and system actions — including system-generated ones (e.g. automatic SLA escalation), which record <code>actorRef: null</code> rather than a human actor.
BR-076	Each audit log entry records the acting user (or none), the action performed, the affected entity type and ID, before/after state, and a timestamp.
BR-077	Audit log entries cannot be deleted or modified once written (insert-only, enforced by never calling update/delete against the model).

Sources: `src/models/Notification.ts`, `src/models/AuditLog.ts`, `src/features/notifications/notifications.integration.test.ts`, `src/features/audit-log/audit-log.integration.test.ts`.

15. Reports

	Rule
BR-078	A generated report record belongs to exactly one creating user.
BR-079	Reports may be generated in multiple formats (PDF, Excel, CSV).
BR-080	Report generation may be filtered by office, college, category, date range, status, and SLA-compliance-only.
BR-081	The filters used to generate a report are recorded alongside the report record itself.
BR-082	The system records a generation timestamp for every report.

Sources: `src/models/GeneratedReport.ts`, `src/app/api/reports/export/route.ts`, `src/features/reports/services/report-data.service.ts`.

Reconciliation note: the original architecture blueprint specified a *stateless* export endpoint ("streams file, no storage of generated reports"). BR-078/082 require the opposite — a persisted record per report — so a `GeneratedReport` collection was added: a row is written when generation starts and updated to `ready` / `failed` with a `downloadUrl` once complete.

16. Authentication

	Rule
BR-083	Every protected page or action requires a valid, authenticated session.
BR-084	Role-based access control is enforced on every protected route, not only authentication.
BR-085	An unauthorized access attempt is rejected — redirected to <code>/login</code> for pages, a 403 JSON response for API routes.
BR-086	Passwords are never stored or compared in plaintext (same guarantee as BR-004, stated from the auth-flow side).
BR-087	A user session expires after a configurable period of inactivity.

Sources: `middleware.ts`, `src/middleware/rbac.ts`, `src/lib/auth.config.ts`, `src/features/auth/services/password.service.ts`, `docs/architecture.md` §5–6.

17. Analytics

	Rule
BR-088	The system shall generate aggregate analytics on complaint volume, resolution time, and SLA compliance.
BR-089	A defined set of KPIs — open/overdue counts, average resolution time, SLA compliance %, category/priority distribution, monthly trend, and average satisfaction rating — shall be available to authorized roles, scoped per role (office/college/institution-wide).
BR-090	Analytics data reflects current data at the time of the query — computed via live aggregation pipelines, not a stale batch export.

Sources: `src/features/analytics/services/analytics.service.ts`, `docs/architecture.md` §11.

18. Security (Role-Scoped Access Enforcement)

	Rule
BR-091	Each role's access to complaint data is scoped to what that role is authorized to see — enforced both at the route level (middleware) and, more importantly, inside every handler (defense in depth).

	Rule
BR-092	A student may access only complaints they submitted themselves.
BR-093	Office staff may access only complaints assigned to their own office.
BR-094	A college dean may access only complaints filed by students belonging to their own college — an oversight scope based on the <i>student's</i> college, independent of which office is handling the case.
BR-095	The QA office has institution-wide read access to complaint data, but no write access to complaint content.
BR-096	An administrator has unrestricted access to all complaint data, all routes, and all admin functions.
BR-097	Every action — not just every page load — must be authorized before it is performed; middleware protects routes, but each handler re-checks role and resource scope, since middleware alone cannot know that "staff A" shouldn't act on office B's complaint just because both are "staff."
BR-098	Data in transit is protected via HTTPS; sensitive data at rest (passwords, reset tokens) is stored hashed, never in recoverable plaintext.

Sources: `src/middleware/rbac.ts`, `src/middleware/rbac.test.ts`, `docs/architecture.md` §6, `src/models/PasswordResetToken.ts`.

19. Data Integrity

	Rule
BR-099	Every reference between collections (a complaint's student, category, office; a rule's target office; etc.) must point to a valid, existing record.
BR-100	Unique identifiers (ticket numbers, office codes, email addresses, student/employee IDs) are immutable once assigned and are never reused — ticket number sequencing uses an atomic <code>\$inc</code> counter specifically so concurrent submissions can never collide.

Sources: `src/models/Counter.ts`, `src/features/complaints/services/ticket-number.service.ts`, model-level `unique` / `required` constraints throughout `src/models/*.ts`.

Configurable Thresholds (Environment Variables)

A subset of the rules above specify "a configurable number/format" rather than a fixed value. Each is wired to exactly one environment variable so the number lives in one place, not scattered across services.

	Requirement	Env var	Default	Consumed by
BR-013	Account lockout after N failed logins, for a configurable duration	<code>AUTH_MAX_FAILED_LOGIN_ATTEMPTS</code> , <code>AUTH_LOCKOUT_DURATION_MINUTES</code>	5, 15	<code>src/features/auth/services/auth.service.ts</code>
BR-087	Sessions expire after a configurable inactivity period	<code>AUTH_SESSION_MAX_AGE_MINUTES</code>	60	<code>src/lib/auth.config.ts</code>
BR-060	Only supported file formats may be uploaded	<code>UPLOAD_ALLOWED_MIME_TYPES</code>	see <code>.env.example</code>	<code>src/app/api/uploads/presign/route.ts</code>
BR-061	Maximum upload size	<code>UPLOAD_MAX_FILE_SIZE_MB</code>	10	<code>src/app/api/uploads/presign/route.ts</code>

	Requirement	Env var	Default	Consumed by
	shall be configurable			
BR-036 / BR-100	Ticket numbers unique, immutable, human-readable (e.g. <code>PARSU-2026-000001</code>)	<code>TICKET_NUMBER_PREFIX</code>	<code>PARSU</code>	<code>src/features/complaints/services/ticket-number.service.ts</code>

Conventions:

- Every var above is read through `src/lib/env.ts` (never `process.env.X` directly in feature code) so a missing/malformed value fails fast at boot with a clear error instead of surfacing as a confusing runtime bug later.
- Numeric vars are validated as positive integers; `UPLOAD_ALLOWED_MIME_TYPES` is validated as a non-empty comma-separated list and exposed to the app as a `string[]`.
- If a future business-rule revision changes a *default*, update it in three places: this table, `.env.example`, and the Zod default in `src/lib/env.ts`. If it changes a *rule ID* or *requirement*, update this document first — it's the source of truth for what each var means.
- Everything else in BR-001–BR-100 is either a fixed structural rule (e.g. BR-021, "every complaint belongs to exactly one category") with no numeric knob, or a rule enforced entirely in code/schema (e.g. BR-045, "deleting complaints is prohibited").

Cross-Reference

- Full traceability (which test covers which rule, and its current test status) lives in [docs/testing/br-traceability.md](#).
- The schema decisions made while reconciling this rule set against the original architecture blueprint are in [docs/schema-reconciliation.md](#).
- The system-level design these rules are implemented against is in [docs/architecture.md](#).